

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический университет»

Электротехнический факультет

Выпускающая кафедра: информационные технологии и автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа № 7

«Функция C++»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнил: студент группы РИС-25-26

Абрамова Валерия Андреевна

Принял: Доц. Полякова О.А.

Пермь 2025

I. Метод половинного деления

1. Постановка задачи

Метод половинного деления

$$0,1x^2 - x \ln x = 0$$

Отрезок, содержащий корень: $[1;2]$

Точное значение: 1,1183

2. Анализ задачи

1. Метод заключается в последовательном уменьшении длины исследуемого интервала путём подбора новых границ. Этот процесс продолжается до тех пор, пока длина отрезка не станет меньше требуемой точности ϵ .

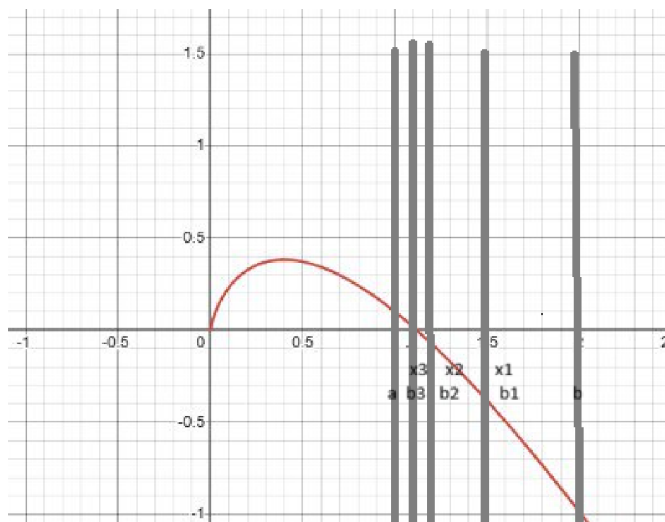
2. Начальная точка x_0 берется на середине интервала $[a;b]$. Разделим отрезок на 2 части: $(a + b) / 2$.

3. Проверяем наличие корня на интервалах $[a, c]$ и $[c, b]$. Если произведение $F(a) * F(c) > 0$, то начало отрезка a переносится в c ($a = c$), иначе конец отрезка b переносится в точку c ($b = c$).

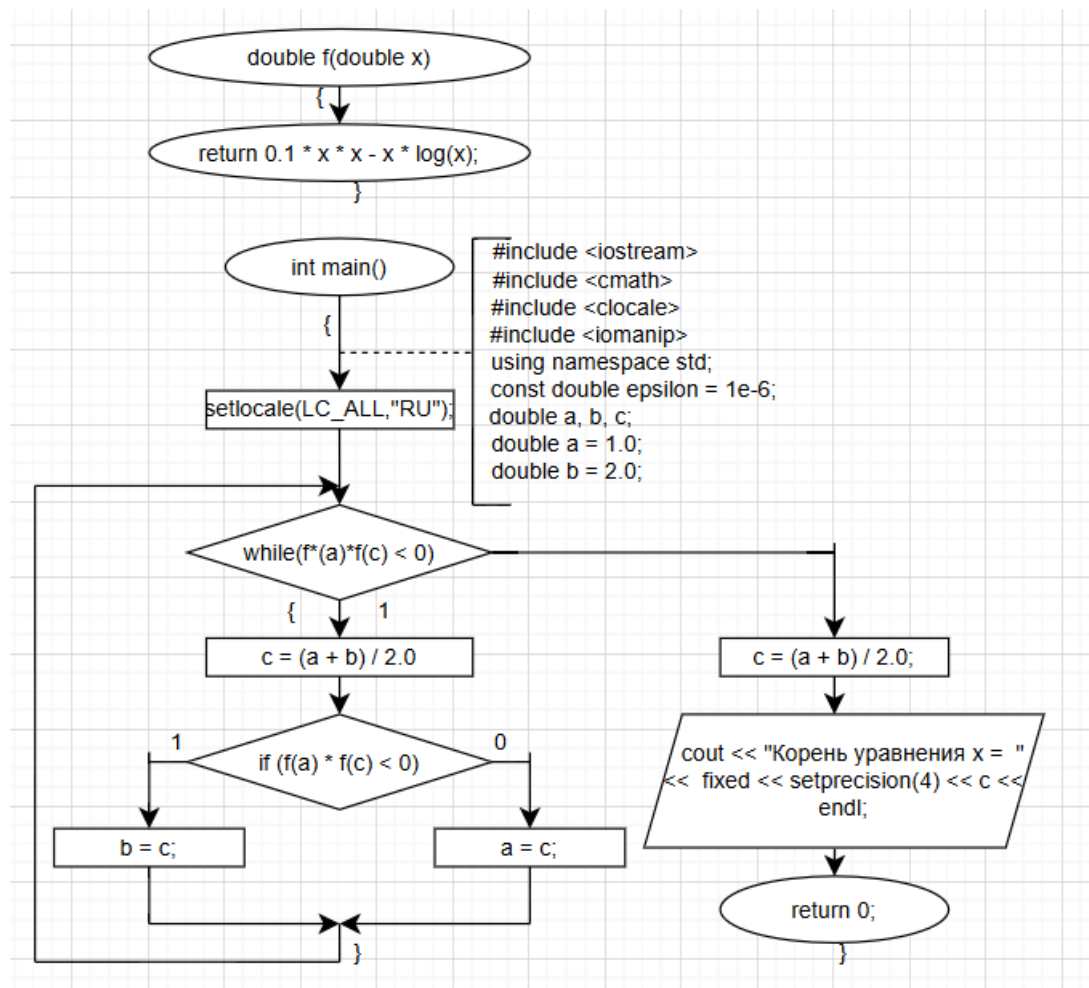
4. Новая точка x_1 выбирается на середине отрезка, в котором есть корень, но при этом, следовательно, мы имеем 2 соседних корня: x_0 и x_1 .

5. Итерационно находятся значения $x_2, x_3, \dots, x_i$, но на каждом шаге сравниваются 2 соседних x_i и $x_{(i+1)}$

$|x_i - x_{(i+1)}| \leq \epsilon$, то есть разность между 2мя соседними x меньше некоторого заданного ϵ , точность вычислений. ϵ крайне мала ($10e-6$) и если $|x_i - x_{(i+1)}|$ меньше ϵ , то x_i и $(i+1)$ попали практически в одну точку, то есть за искомый корень можно брать любой из них. Следовательно программа выполняется пока $|x_i - x_{(i+1)}| \leq \epsilon$.



3. Блок-схема



4. Код

```
1  #include <iostream>
2  #include <cmath>
3  #include <locale>
4  #include <iomanip>
5  using namespace std;
6
7  const double epsilon = 1e-6;
8
9  double f(double x) {
10     return 0.1 * x * x - x * log(x);
11 }
12
13 int main()
14 {
15     setlocale(LC_ALL, "RU");
16     double a, b, c;
17     a = 1.0;
18     b = 2.0;
19     while (fabs(b - a) > epsilon) {
20         c = (a + b) / 2.0;
21
22         if (f(a) * f(c) < 0)
23             b = c;
24         else
25             a = c;
26     }
27     c = (a + b) / 2.0;
28     cout << "Корень уравнения x = " << fixed << setprecision(4) << c << endl;
29     return 0;
30 }
```

Таблица расчётов

```
1  #include <iostream>
2  #include <cmath>
3  #include <locale>
4  #include <iomanip>
5
6  using namespace std;
7
8  const double epsilon = 1e-6;
9
10 double f(double x) {
11     return 0.1 * x * x - x * log(x);
12 }
13 int main()
14 {
15     setlocale(LC_ALL, "RU");
16     double a, b, c;
17     a = 1.0;
18     b = 2.0;
19     while (fabs(b - a) > epsilon) {
20         c = (a + b) / 2.0;
21         cout << "a: " << a << " b: " << b << " x: " << c << " f(a): " << f(a) << " f(x): " << f(c) << " f(a) * f(x): " << f(a) * f(c) << endl;
22         if (f(a) + f(c) < 0)
23             b = c;
24         else
25             a = c;
26     }
27     c = (a + b) / 2.0;
28     return 0;
29 }
```

```
Консоль отладки Microsoft Vi
a: 1 b: 2 x: 1.5 f(a) :-0.383198 f(a)*f(x): -0.0383198
a: 1 b: 1.5 x: 1.25 f(a) :-0.122679 f(a)*f(x): -0.0122679
a: 1 b: 1.25 x: 1.125 f(a) :-0.00594342 f(a)*f(x): -0.000594342
a: 1 b: 1.125 x: 1.0625 f(a) :0.048477 f(a)*f(x): 0.0048477
a: 1.0625 b: 1.125 x: 1.09375 f(a) :0.0216156 f(a)*f(x): 0.00104786
a: 1.09375 b: 1.125 x: 1.10938 f(a) :0.00792172 f(a)*f(x): 0.000171233
a: 1.10938 b: 1.125 x: 1.11719 f(a) :0.00101037 f(a)*f(x): 8.00384e-06
a: 1.11719 b: 1.125 x: 1.12109 f(a) :-0.00246125 f(a)*f(x): -2.48676e-06
a: 1.11719 b: 1.12109 x: 1.11914 f(a) :-0.000724117 f(a)*f(x): -7.31623e-07
a: 1.11719 b: 1.11914 x: 1.11816 f(a) :0.000143456 f(a)*f(x): 1.44943e-07
a: 1.11816 b: 1.11914 x: 1.11865 f(a) :-0.000290248 f(a)*f(x): -4.16377e-08
a: 1.11816 b: 1.11865 x: 1.11841 f(a) :-7.33752e-05 f(a)*f(x): -1.05261e-08
a: 1.11816 b: 1.11841 x: 1.11829 f(a) :3.50455e-05 f(a)*f(x): 5.02748e-09
a: 1.11829 b: 1.11841 x: 1.11835 f(a) :-1.91636e-05 f(a)*f(x): -6.71597e-10
a: 1.11829 b: 1.11835 x: 1.11832 f(a) :7.94125e-06 f(a)*f(x): 2.78305e-10
a: 1.11832 b: 1.11835 x: 1.11833 f(a) :-5.61109e-06 f(a)*f(x): -4.45591e-11
a: 1.11832 b: 1.11833 x: 1.11832 f(a) :1.1651e-06 f(a)*f(x): 9.25237e-12
a: 1.11832 b: 1.11833 x: 1.11833 f(a) :-2.22299e-06 f(a)*f(x): -2.59001e-12
a: 1.11832 b: 1.11833 x: 1.11833 f(a) :-5.28943e-07 f(a)*f(x): -6.16272e-13
a: 1.11832 b: 1.11833 x: 1.11833 f(a) :3.18008e-07 f(a)*f(x): 3.70595e-13
```

Количество итераций и время выполнения

```
ConsoleApplication23 (Глобальная область)
1  #include <iostream>
2  #include <cmath>
3  #include <locale>
4  #include <iomanip>
5  using namespace std;
6
7  const double epsilon = 1e-6;
8
9  double f(double x) {
10     return 0.1 * x * x - x * log(x);
11 }
12
13 int main()
14 {
15     setlocale(LC_ALL, "RU");
16     double a, b, c;
17     a = 1.0;
18     b = 2.0;
19     int iter = 0;
20     while (fabs(b - a) > epsilon) {
21         iter++;
22         c = (a + b) / 2.0;
23
24         if (f(a) * f(c) < 0)
25             b = c;
26         else
27             a = c;
28     }
29     c = (a + b) / 2.0;
30     cout << "Количество итераций: " << iter << endl;
31     return 0;
32 }
```

Консоль отладки Microsoft Visual Studio

Количество итераций: 20

C:\Users\Valeria\source\repos\ConsoleApplication23\bin\Debug\ConsoleApplication23.exe: Код ошибки 0 (0x0).

Чтобы автоматически закрывать консоль при выходе из отладки, нажмите Ctrl+F5.

```
ConsoleApplication23 (Глобальная область)
1  #include <iostream>
2  #include <cmath>
3  #include <locale>
4  #include <iomanip>
5  #include <chrono>
6
7  using namespace std;
8
9  const double epsilon = 1e-6;
10
11 double f(double x) {
12     return 0.1 * x * x - x * log(x);
13 }
14
15 int main()
16 {
17     setlocale(LC_ALL, "RU");
18     auto start = chrono::high_resolution_clock::now();
19     double a, b, c;
20     a = 1.0;
21     b = 2.0;
22     while (fabs(b - a) > epsilon) {
23         c = (a + b) / 2.0;
24
25         if (f(a) * f(c) < 0)
26             b = c;
27         else
28             a = c;
29     }
30     c = (a + b) / 2.0;
31     auto end = chrono::high_resolution_clock::now();
32     chrono::duration<double> duration = end - start;
33     cout << "Время выполнения: " << duration.count() << " секунд" << endl;
34     return 0;
35 }
```

Консоль отладки Microsoft Visual Studio

Время выполнения: 1.25e-05 секунд

C:\Users\Valeria\source\repos\ConsoleApplication23\bin\Debug\ConsoleApplication23.exe: Код ошибки 0 (0x0).

II. Метод Ньютона

1. Постановка задачи

Метод Ньютона

$$0,1x^2 - x \ln x = 0$$

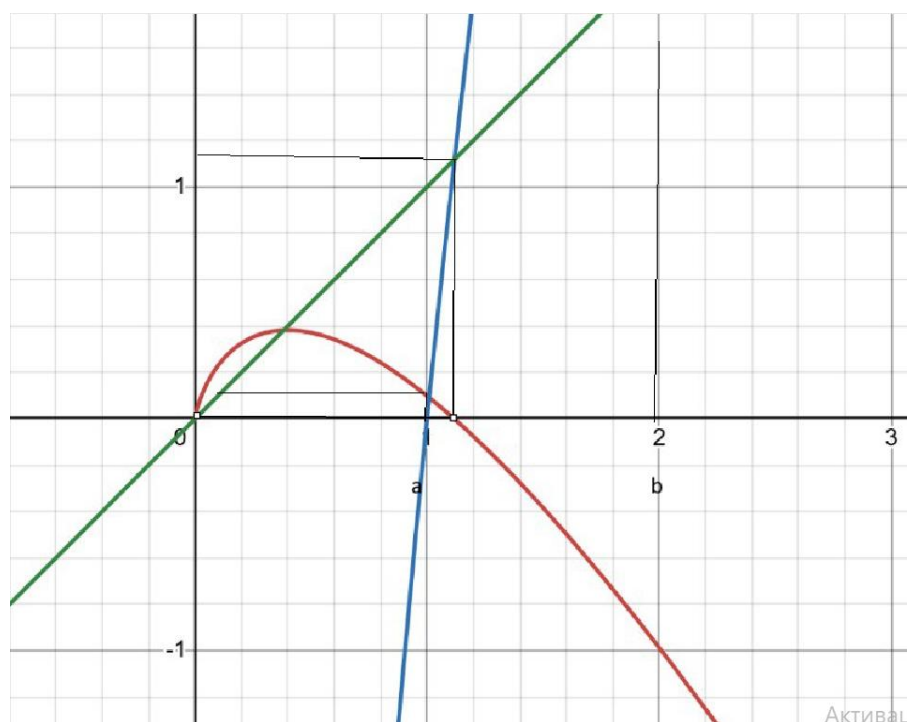
Отрезок, содержащий корень: [1;2]

Точное значение: 1,1183

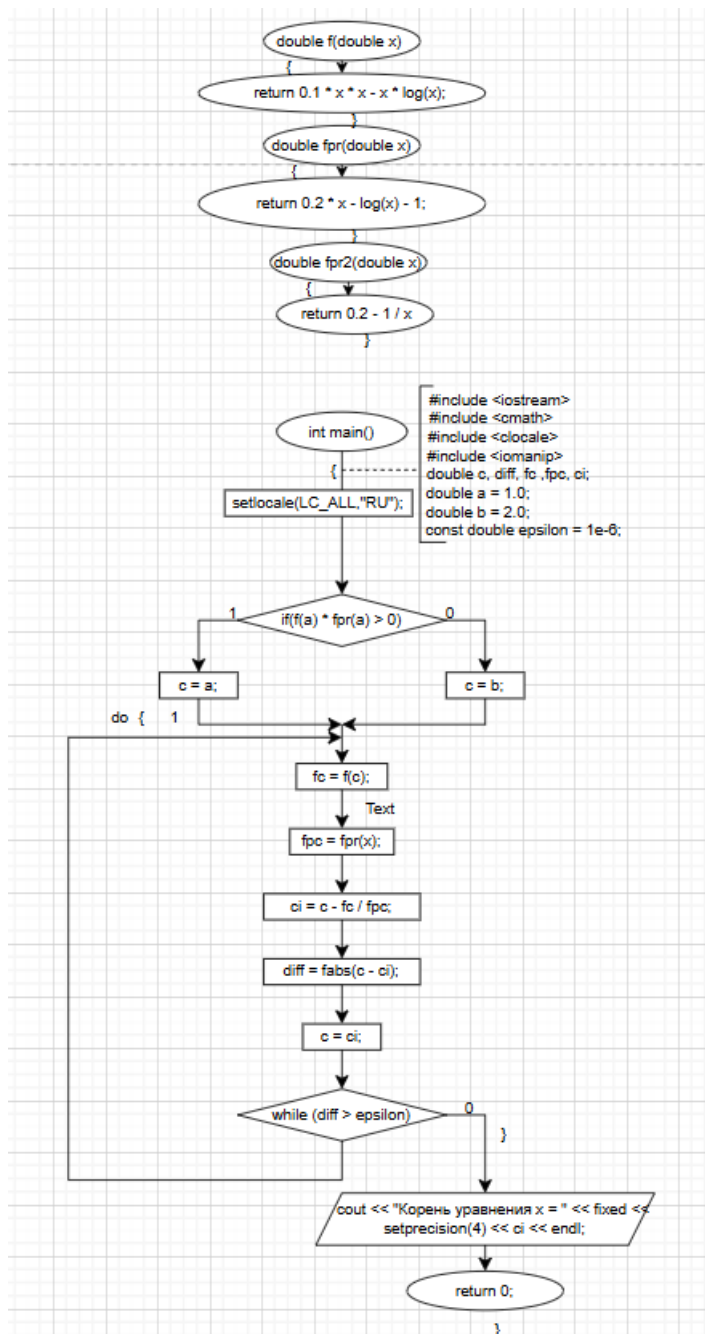
2. Анализ задачи:

1. Начальная точка x_0 берется на середине интервала $[a;b]$. Разделим отрезок на 2 части: $(a + b) / 2$.
2. Проверка корня на интервале. Если условие $f(a) * f(b) < 0$, то корень на интервале есть.
3. Необходимо найти первую и вторую производную то $f(x)$, которая отвечает за монотонность и непрерывность функции. Если $f'(x) < 0$ или $f'(x) > 0$ для всех x на $[a;b]$, то тогда функция непрерывна и монотонна. Далее находим вторую производную, которая отвечает за выпуклость и вогнутость функции.
4. Определение подхода:
Подход слева, если $f(a) * f''(a) > 0$
Подход справа, если $f(b) * f''(b) > 0$
5. Проверяем $|x_0 - x_1| < \epsilon$. Если условие выполняется, то корень x_1 , иначе выходим из цикла.
6. В точке $f(x_0)$ проводится касательная к графику функции. Точка пересечения – x_1 .
$$x = x_0 - (f(x_0) / f'(x_0))$$

3. Блок-схема



Активал



4. Код


```
ConsoleApplication24
1  #include <iostream>
2  #include <cmath>
3  #include <locale>
4  #include <iomanip>
5  using namespace std;
6
7  const double epsilon = 1e-6;
8
9  double f(double x) {
10     return 0.1 * x * x - x * log(x);
11 }
12
13 double fpr(double x) {
14     return 0.2 * x - log(x) - 1;
15 }
16
17 double fpr2(double x) {
18     return 0.2 - 1 / x;
19 }
20
21 int main()
22 {
23
24     setlocale(LC_ALL, "RU");
25     double c, diff, fc, fpc, ci;
26     double a = 1.0;
27     double b = 2.0;
28     if (f(a) * fpr2(a) > 0) {
29         c = a;
30     }
31     else {
32         c = b;
33     }
34
35     do {
36         fc = f(c);
37         fpc = fpr(c);
38         ci = c - fc / fpc;
39         diff = fabs(c - ci);
40         c = ci;
41     } while (diff > epsilon);
42
43     cout << "Корень уравнения x = " << fixed << setprecision(4) << ci << endl;
44     return 0;
45 }
46
47
```

Консоль отладки Microsoft Vi

Корень уравнения x = 1.1183

C:\Users\Valeria\source\repos\Conso

Количество итераций и время выполнения

```

1  #include <iostream>
2  #include <cmath>
3  #include <locale>
4  #include <iomanip>
5  #include <chrono>
6  using namespace std;
7
8  const double epsilon = 1e-6;
9
10 double f(double x) {
11     return 0.1 * x * x - x * log(x);
12 }
13
14 double fpr(double x) {
15     return 0.2 * x - log(x) - 1;
16 }
17
18 double fpr2(double x) {
19     return 0.2 - 1 / x;
20 }
21
22 int main()
23 {
24     setlocale(LC_ALL, "RU");
25     auto start = chrono::high_resolution_clock::now();
26     double c, diff, fc, fpc, ci;
27     double a = 1.0;
28     double b = 2.0;
29     if (f(a) * fpr2(a) > 0) {
30         c = a;
31     }
32     else {
33         c = b;
34     }
35
36     do {
37         fc = f(c);
38         fpc = fpr(c);
39         ci = c - fc / fpc;
40         diff = fabs(c - ci);
41         c = ci;
42     } while (diff > epsilon);
43
44     auto end = chrono::high_resolution_clock::now();
45     chrono::duration<double> duration = end - start;
46     cout << "Время выполнения: " << duration.count() << " секунд" << endl;
47     return 0;
48 }
49
50

```



Консоль отладки Microsoft Vi



Время выполнения: 1.8e-05 секунд

C:\Users\Valeria\source\repos\ConsoleApplicati

```
ConsoleApplication24
1  #include <iostream>
2  #include <cmath>
3  #include <locale>
4  #include <iomanip>
5  using namespace std;
6
7  const double epsilon = 1e-6;
8
9  double f(double x) {
10     return 0.1 * x * x - x * log(x);
11 }
12
13 double fpr(double x) {
14     return 0.2 * x - log(x) - 1;
15 }
16
17 double fpr2(double x) {
18     return 0.2 - 1 / x;
19 }
20
21 int main()
22 {
23     setlocale(LC_ALL, "RU");
24     double c, diff, fc, fpc, ci;
25     double a = 1.0;
26     double b = 2.0;
27     int iter = 0;
28     if (f(a) * fpr2(a) > 0) {
29         c = a;
30     }
31     else {
32         c = b;
33     }
34
35     do {
36         iter++;
37         fc = f(c);
38         fpc = fpr(c);
39         ci = c - fc / fpc;
40         diff = fabs(c - ci);
41         c = ci;
42     } while (diff > epsilon);
43
44     cout << "Количество итераций: " << iter << endl;
45     return 0;
46 }
47
48
49
```

Консоль отладки Microsoft Vi × + ▾

Количество итераций: 5

C:\Users\Valeria\source\repos\ConsoleApplication24\bin\Debug\ConsoleApplication24.exe
кодом 0 (0x0).

Таблица расчетов и время выполнения

```

ConsoleApplication24 (Глобальная область
1  #include <iostream>
2  #include <cmath>
3  #include <locale>
4  #include <iomanip>
5  using namespace std;
6
7  const double epsilon = 1e-6;
8
9  double f(double x) {
10     return 0.1 * x * x - x * log(x);
11 }
12
13 double fpr(double x) {
14     return 0.2 * x - log(x) - 1;
15 }
16
17 double fpr2(double x) {
18     return 0.2 - 1 / x;
19 }
20
21 int main()
22 {
23     setlocale(LC_ALL, "RU");
24
25     double c, diff, fc, fpc, ci;
26     double a = 1.0;
27     double b = 2.0;
28     if (f(a) * fpr2(a) > 0) {
29         c = a;
30     }
31     else {
32         c = b;
33     }
34
35     do {
36         fc = f(c);
37         fpc = fpr(c);
38         ci = c - fc / fpc;
39         diff = fabs(c - ci);
40         c = ci;
41
42         cout << "x = " << c << " f(x) = " << fc << " f'(x) = " << fpc << " abs(x - xi) = " << diff << endl;
43     } while (diff > epsilon);
44
45     return 0;
46 }
47
48

```

```

x = 1.23729 f(x) = -0.986294 f'(x) = -1.29315 abs(x - xi) = 0.762709
x = 1.12298 f(x) = -0.110361 f'(x) = -0.965466 abs(x - xi) = 0.114308
x = 1.11833 f(x) = -0.00414417 f'(x) = -0.891392 abs(x - xi) = 0.0046491
x = 1.11833 f(x) = -7.47542e-06 f'(x) = -0.888173 abs(x - xi) = 8.41662e-06
x = 1.11833 f(x) = -2.4588e-11 f'(x) = -0.888167 abs(x - xi) = 2.76839e-11

```

C:\Users\Valeria\source\repos\ConsoleApplication22\x64\Debug\ConsoleApplication22.exe (n
кодом 0 (0x0)).

III. Метод итераций

1. Постановка задачи

Итерационный метод

$$0,1x^2 - x \ln x = 0$$

Отрезок, содержащий корень: [1;2]

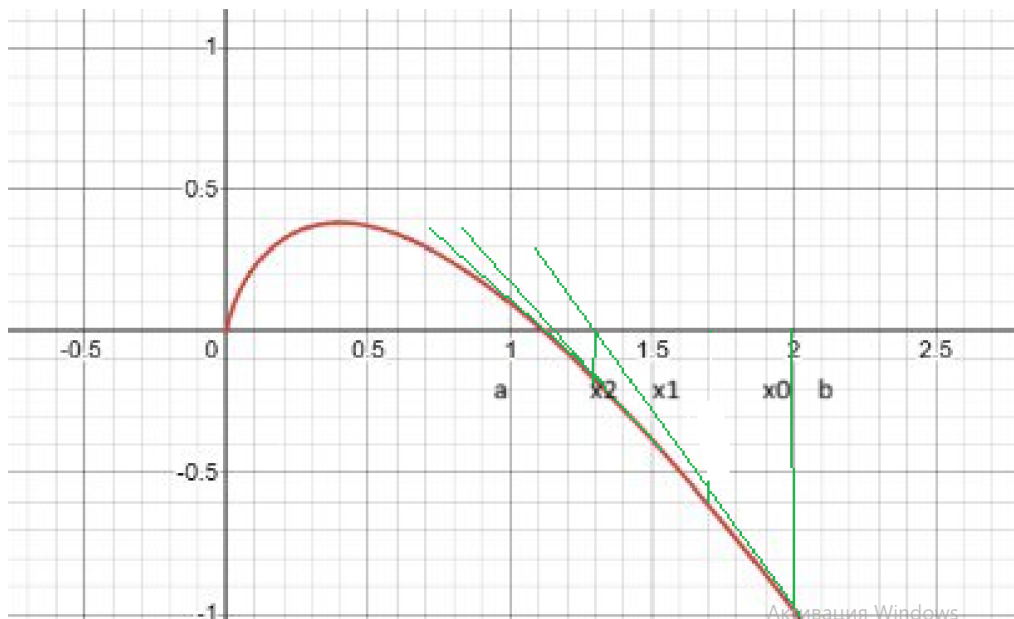
Точное значение: 1,1183

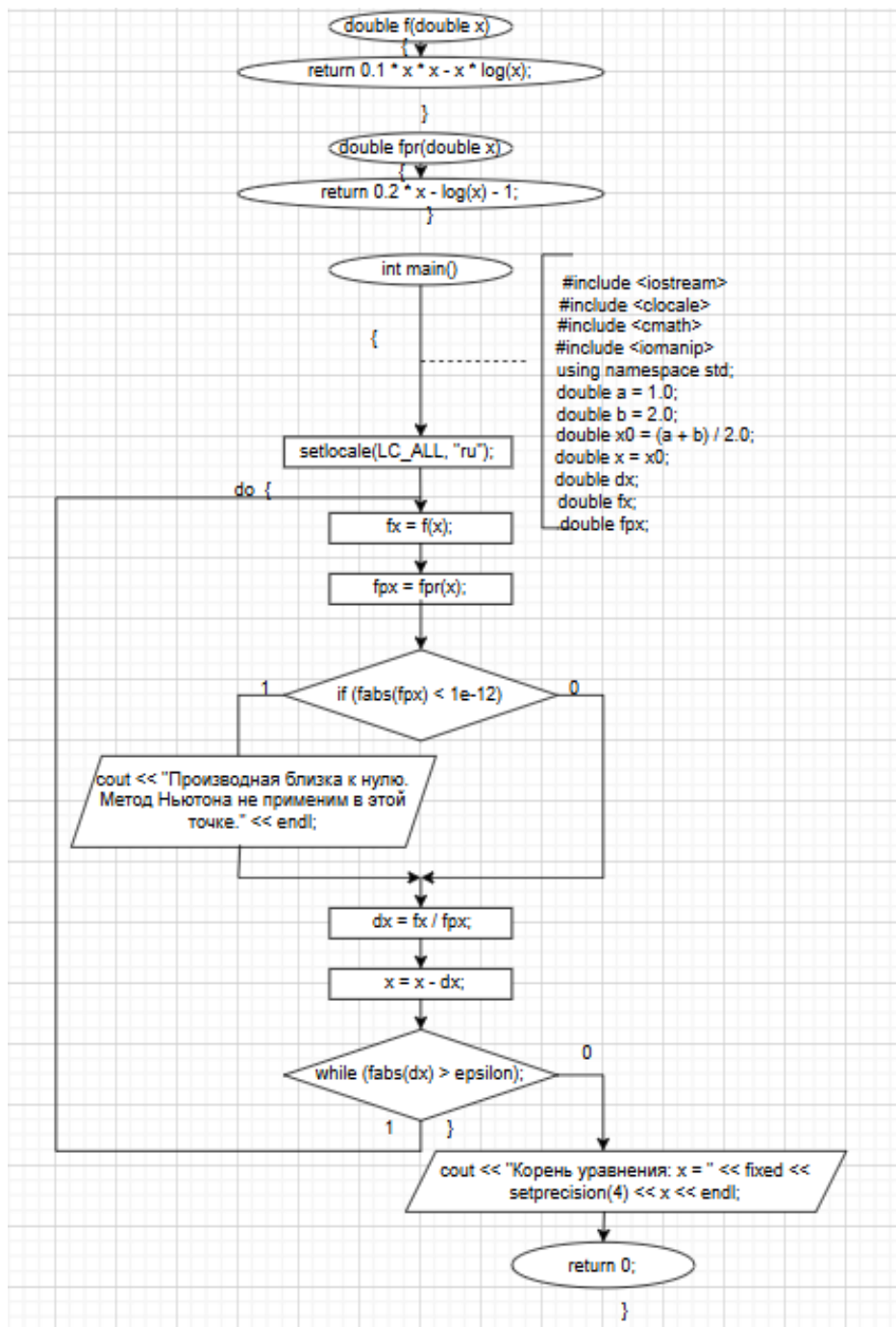
2. Анализ задачи

1. Проверяем наличие корня на отрезке [a; b] – убеждаемся, что произведение значений функции на концах отрицательное: $f(a) \times f(b) < 0$

2. Оцениваем непрерывность – на заданном интервале график функции не имеет разрывов, функция $f(x)$ везде определена
3. Определяем монотонность – производная $f'(x)$ сохраняет знак на $[a; b]$ (всегда положительна или всегда отрицательна), что обеспечивает единственность корня
4. Преобразуем уравнение к виду $x = \varphi(x)$, подбирая такое преобразование, чтобы получить явное выражение для x
5. Выбираем начальное приближение x_i , учитывая границы интервала и поведение функции
6. Выполняем итерации по формуле: $x_i = \varphi(x_{i-1})$, последовательно уточняя значение
7. Итерационный процесс продолжаем до выполнения условия $|x_i - x_{(i-1)}| \leq \epsilon$ после чего полученное значение принимаем за корень

3. Блок-схема





4.Код

```

#include <iostream>
#include <locale>
#include <cmath>
#include <iomanip>
using namespace std;

double f(double x) {
    return 0.1 * x * x - x * log(x);
}

double fpr(double x) {
    return 0.2 * x - log(x) - 1;
}

int main() {
    setlocale(LC_ALL, "ru");
    double a = 1.0;
    double b = 2.0;
    const double epsilon = 1e-6;
    double x0 = (a + b) / 2.0;
    double x = x0;
    double dx;

    do {
        double fx = f(x);
        double fpx = fpr(x);
        if (fabs(fpx) < 1e-12) {
            cout << "Производная близка к нулю. Метод Ньютона не применим в этой точке." << endl;
            break;
        }
        dx = fx / fpx;
        x = x - dx;
    } while (fabs(dx) > epsilon);

    cout << "Корень уравнения: x = " << fixed << setprecision(4) << x << endl;
    return 0;
}

```

Таблица расчетов

Количество итераций и время выполнения

ConsoleApplication26.cpp

ConsoleApplication26

(Глобал

```
1  #include <iostream>
2  #include <locale>
3  #include <cmath>
4  #include <iomanip>
5  using namespace std;
6
7  const double epsilon = 1e-6;
8
9  double f(double x) {
10     return 0.1 * x * x - x * log(x);
11 }
12
13 double fpr(double x) {
14     return 0.2 * x - log(x) - 1;
15 }
16
17 int main() {
18     setlocale(LC_ALL, "ru");
19     double a = 1.0;
20     double b = 2.0;
21     double x0 = (a + b) / 2.0;
22     double x = x0;
23     double dx;
24     int iter = 0;
25
26     do {
27         iter++;
28
29         double fx = f(x);
30         double fpx = fpr(x);
31
32         if (fabs(fpx) < 1e-12) {
33             cout << "Производная близка к нулю. Метод Ньютона не применим в этой точке." << endl;
34         }
35
36         dx = fx / fpx;
37         x = x - dx;
38     } while (fabs(dx) > epsilon);
39
40     cout << "Количество итераций: " << iter << endl;
41
42     return 0;
43 }
44
```

Консоль отладки Microsoft Vi

+ v

Количество итераций: 4

C:\Users\Valeria\source\repos\ConsoleApplication26\x64\Deb
спом 0 (0x0)


```
1  #include <iostream>
2  #include <locale>
3  #include <cmath>
4  #include <iomanip>
5  #include <chrono>
6  using namespace std;
7  using namespace std::chrono;
8
9  const double epsilon = 1e-6;
10
11 double f(double x) {
12     return 0.1 * x * x - x * log(x);
13 }
14
15 double fpr(double x) {
16     return 0.2 * x - log(x) - 1;
17 }
18
19 int main() {
20     setlocale(LC_ALL, "ru");
21
22     auto start = high_resolution_clock::now();
23
24     double a = 1.0;
25     double b = 2.0;
26     double x0 = (a + b) / 2.0;
27     double x = x0;
28     double dx;
29
30     do {
31         double fx = f(x);
32         double fpx = fpr(x);
33         if (fabs(fpx) < 1e-12) {
34             cout << "Производная близка к нулю. Метод Ньютона не применим в этой точке." << endl;
35         }
36         dx = fx / fpx;
37         x = x - dx;
38     } while (fabs(dx) > epsilon);
39     auto end = high_resolution_clock::now();
40
41     duration<double> duration_sec = end - start;
42
43     cout << "Время выполнения: " << scientific << setprecision(6) << duration_sec.count() << " секунд" << endl;
44
45     return 0;
46 }
```

Консоль отладки Microsoft Visual Studio

Время выполнения: 1.040000e-05 секунд

C:\Users\Valeria\source\repos\ConsoleApplication26\x64\Debug\ConsoleApplication26.exe: Завершено выполнение программы. Код возврата: 0 (0x0).

Заключение:

У каждого метода есть как плюсы, так и минусы.

1. Из всех способов самый медленный – Метод Ньютона, а самый быстрый – Метод простых итераций
2. Меньше всего итераций происходит в Методе Ньютона, а больше всего в Методе половинного деления
3. По сложности код Метод половинного деления оказался самым простым, а самым сложным Метод простых итераций